

ARY_GRS_001_PoC

（一）展示流程

1. 首次准备

1. 首次准备

在仓库根目录执行：

```
npm install
cp .env.example .env # 或用 copy / Copy-Item
npx prisma migrate dev --name init
npx prisma generate
npm run db:seed
```

如果你之前已经初始化过数据库，只想把演示数据重置回默认状态，执行：

```
npm run db:seed
```

根据种子初始化数据库

2. 终端 1：作为 Organizer 登录 ARY

2. 终端 A：启动 ARY Web 应用

在仓库根目录执行：

```
npm run dev
```

启动后访问：

- 主界面： <http://localhost:3000>
- Audience 视图： <http://localhost:3000/audience>

```
○ (base) PS E:\CodeBuddy\ARY-for-ARY> npm run dev

> next-scaffold@0.1.0 dev
> next dev

✓ Console Ninja extension is connected to Next.js, see https://tinyurl.com/2vt8jxzw
▲ Next.js 16.2.7 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.56.1:3000
- Environments: .env
✓ Ready in 1078ms
△ Console Ninja Turbopack support is a PRO feature, currently available to community users while in pre view
```

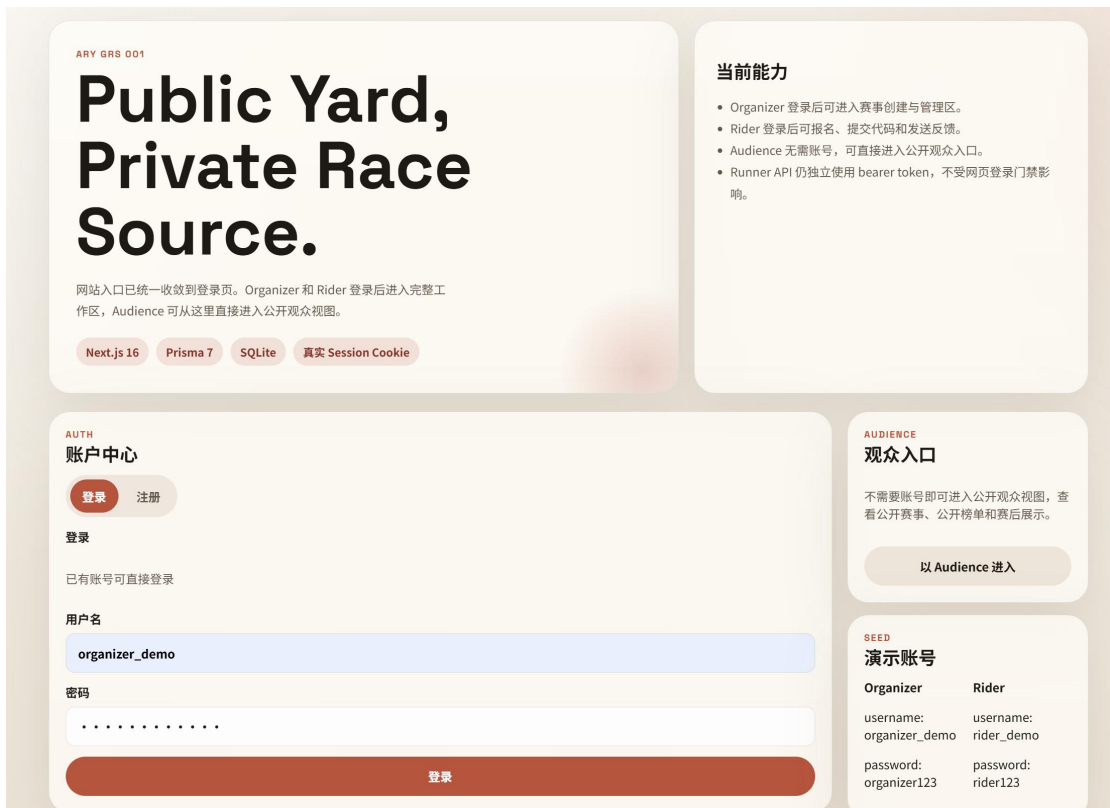
去到 <http://localhost:3000>，登录 Organizer 账号

如果已经登录了 Organizer 就跳过登录，如果登录了 Rider 请自行点击退出登录

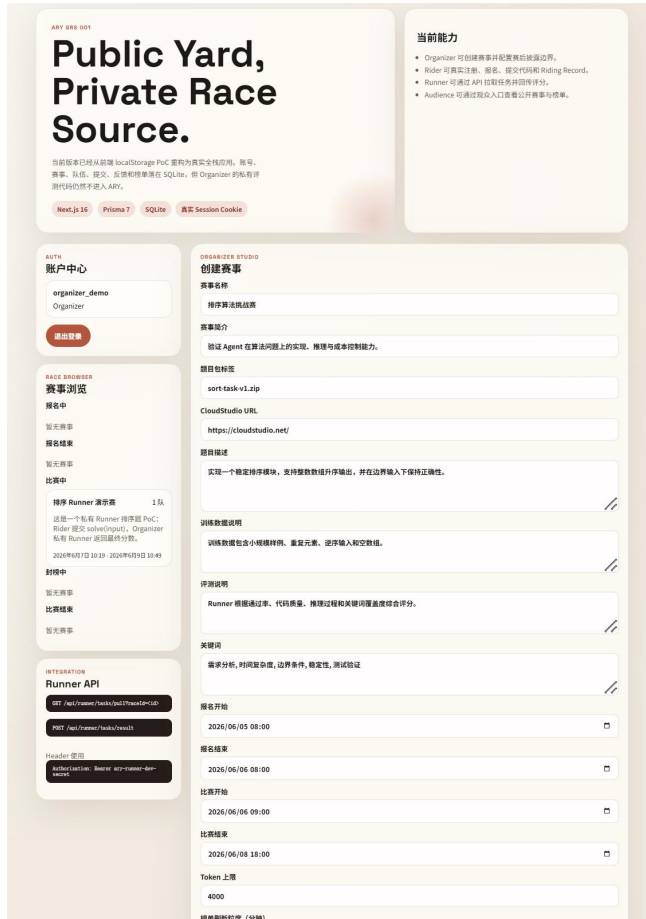
演示账号：username: [organizer_demo](#) + password: [organizer123](#)

username: [rider_demo](#) + password: [rider123](#)

若输入密码错误请刷新网页即可



登录后网页大致是：



创建赛事：补充完相关信息后点击最下方的创建赛事即可
目前未实现真实的上传 zip 形式，只是简单输入题目包标签

ORGANIZER STUDIO

创建赛事

赛事名称

排序算法挑战赛demo2

赛事简介

验证 Agent 在算法问题上的实现、推理与成本控制能力。

题目包标签

sort-task-v1.zip

CloudStudio URL

https://cloudstudio.net/

题目描述

实现一个稳定排序模块，支持整数数组升序输出，并在边界输入下保持正确性。

训练数据说明

训练数据包含小规模样例、重复元素、逆序输入和空数组。

创建后就可以看到相应比赛出现在下方：

比赛中

排序算法挑战赛demo2

0 支队伍24h 提交冷却Top 3 Highlight

验证 Agent 在算法问题上的实现、推理与成本控制能力。

PUBLIC PROJECTION

公开规则

题目包

sort-task-v1.zip

CloudStudio

https://cloudstudio.net/

报名时间

2026年6月5日 08:00 - 2026年6月6日 08:00

比赛时间

2026年6月6日 09:00 - 2026年6月8日 18:00

评测说明

Runner 根据通过率、代码质量、推理过程和关键词覆盖度综合评分。

关键词

["需求分析" / "时间复杂度" / "边界条件" / "稳定性" / "测试验证"]

LEADERBOARD

公开榜单

尚未同步榜单。

BOUNDARY

题目与赛后披露

实现一个稳定排序模块，支持整数数组升序输出，并在边界输入下保持正确性。

训练数据：训练数据包含小规模样例、重复元素、逆序输入和空数组。

• Organizer 评论：公开

• Rider 代码：公开

• Top Highlights：公开前 3 条

SHOWCASE

赛后展示

尚未发布赛后展示。

3. 终端 2: Organizer 在自己环境下创建私有 runner

3. 终端 B: 启动 Organizer 私有 Runner

在 `organizer_demo/runner_demo` 目录执行:

```
cd organizer_demo/runner_demo
cp .env.example .env # 或用 copy / Copy-Item
npm install
npm run start
```

正常情况下你会看到类似日志:

```
[runner_demo] polling race race_sort_demo every 2000ms on http://localhost:3000
No queued tasks for race race_sort_demo.
```

这表示私有 Runner 已经开始轮询 ARY。

```
● (base) PS E:\CodeBuddy\ARY-for-ARY> cd organizer_demo/runner_demo
● (base) PS E:\CodeBuddy\ARY-for-ARY\organizer_demo\runner_demo> cp .env.example .env
● (base) PS E:\CodeBuddy\ARY-for-ARY\organizer_demo\runner_demo> npm install

up to date, audited 7 packages in 1s

found 0 vulnerabilities
○ (base) PS E:\CodeBuddy\ARY-for-ARY\organizer_demo\runner_demo> npm run start

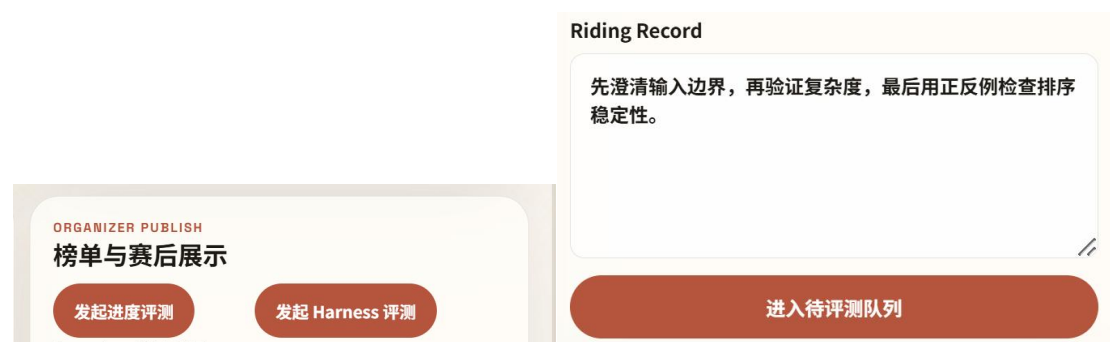
> organizer-runner-demo@0.1.0 start
> node --env-file=.env --import tsx src/index.ts

[runner_demo] polling race race_sort_demo every 2000ms on http://localhost:3000
No queued tasks for race race_sort_demo.
No queued tasks for race race_sort_demo.
```

这表示 Runner 已经在不断读取 ARY 的任务列表

目前, 对于“进度测评”和“代码测试”共用一个 runner

其中“代码测试”由 Organizer 主动点击[发起进度评测](#)实现, “代码测试”由 Rider 自行点击[进入带评测队列](#)实现:



4. 回到终端 1: 退出 Organizer 账号, 登录 Rider 账号

在下方找到想要加入的比赛 (为了展示我们选择刚刚创建的 demo2 赛事)

比赛中

排序算法挑战赛demo2

0 支队伍

24h 提交冷却

Top 3 Highlight

验证 Agent 在算法问题上的实现、推理与成本控制能力。

PUBLIC PROJECTION

公开规则

题目包

CloudStudio

sort-task-v1.zip

<https://cloudstudio.net/>

报名时间

比赛时间

2026年6月5日 08:00 - 2026年6月6日 08:00

2026年6月6日 09:00 - 2026年6月8日 18:00

评测说明

关键词

Runner 根据通过率、代码质量、推理过程和关键词覆盖度综合评分。

["需求分析" / "时间复杂度" / "边界条件" / "稳定性" / "测试验证"]

LEADERBOARD

公开榜单

尚未同步榜单。

在赛事报名栏目填写信息：

RIDER

报名参赛

队伍名称

组员列表

提交报名

由于刚刚的 demo2 项目比赛，报名时间和比赛开始时间有点问题，下面的演示我们采用第一版的“排序算法挑战赛”来展示：
该比赛我们已经提前报名进入：

RIDER

当前队伍

排序演示队

- rider_demo
- 队友 A

当 Rider 主动提交代码，请求测试：

SUBMISSION

提交代码与 Riding Record

代码文件名

solution.ts

Record 文件名

riding-record.txt

Agent 类型

OpenAI

Token 消耗

1200

代码内容

```
export function solve(input: number[]) {  
  return [...input].sort((a, b) => a - b);  
}
```

Riding Record

先澄清输入边界，再验证复杂度，最后用正反例检查排序稳定性。

进入待评测队列

点击“进入待测评队列”后，可在下方“提交流程状态”看到相关信息：

RUNNER QUEUE

提交流程状态

队伍	任务类型	状态	提交 ID	总分	时间
排序演示队	提交测试	成功	cmq37jtku0001wcfvbk33ecoz	100	2026年6月7日 11:13

本来已经状态是“排队中”，然后由 Organizer 的 runner 拉取评分后返回才变成“成功”，但是 runner 太快了（你可以通过不刷新界面看到“排队中”）
下面是刚刚的终端 2 的 Organizer 的私有 runner 的接受证据：

```
No queued tasks for race race_sort_demo.  
Processed submission_test task cmq37jnle0003wcfvityd441.  
No queued tasks for race race_sort_demo.
```

可以看到 runner 成功接收，并在 Organizer 自己环境里进行了测试

最终仅返回分数给 ARY

5. 在终端 1 退出 Rider 账号，登录 Organizer 账号，展示“进度评测”功能
Orgnizer 主动点击[发起进度测评](#)

ORGANIZER PUBLISH

榜单与赛后展示

发起进度评测

发起 Harness 评测

Organizer 赛后总评

提交一个 TypeScript 排序解法，等待私有 Runner 完成评分，再使用现有发榜按钮刷新公开榜单。

保存总评

☒ 公开训练数据

☒ 公开 Organizer 评论

☒ 展示 Top Highlights

☒ 公开 Rider 代码

Highlight 数量

3

保存展示选项

一键清空比赛

然后可以看到下方提交流程状态中：

RUNNER QUEUE

提交流程状态

队伍	任务类型	状态	提交 ID	总分	时间
排序演示队	进度评测	排队中	cmq37jnku0001wcfvbk33ecoz	-	2026年6月7日 11:20
排序演示队	提交测试	成功	cmq37jnku0001wcfvbk33ecoz	100	2026年6月7日 11:13

然后看终端 2 的 runner：

```
No queued tasks for race race_sort_demo.  
Processed progress_eval task cmq37s9we0004wcfvbkpmqscxm.  
No queued tasks for race race_sort_demo.
```

然后刷新 ARY 界面：

RUNNER QUEUE

提交流程状态

队伍	任务类型	状态	提交 ID	总分	时间
排序演示队	进度评测	成功	cmq37jnku0001wcfvbk33ecoz	100	2026年6月7日 11:20
排序演示队	提交测试	成功	cmq37jnku0001wcfvbk33ecoz	100	2026年6月7日 11:13

可以看到 runner 已经主动读取了 Rider 的数据，进行进度评价后返回
此时若我们再去到 Rider 账号里，可以看到下面的提交流程状态也同步更新了：

RUNNER QUEUE					
提交流程状态					
队伍	任务类型	状态	提交 ID	总分	时间
排序演示队	进度评测	成功	cmq37jnku0001wcfvbk33ecoz	100	2026年6月7日 11:20
排序演示队	提交测试	成功	cmq37jnku0001wcfvbk33ecoz	100	2026年6月7日 11:13

并且，根据进度测评的结果，我们设置了榜单：

LEADERBOARD						
公开榜单						
排名	队伍	总分	任务	Token	对话	Agent
1	排序演示队	100	-	-	-	OpenAI

6. 总结

至此，我们已经演示了 ARY 从 Organizer 创建比赛、Organizer 配置私有 runner、Rider 报名、Rider 提交代码测试、Organizer 主动发起进度测评的流程。

另外有的额外功能由于不涉及 GRS_001 的要求，我们不做展示。

（二）安全问题

（1）Rider 的数据不值钱，所以我们没有针对 Rider 进行数据保护，Organizer 可以随时拉取 Rider 的数据/Harness 记录进行操作

（2）Organizer 的数据经过了严密保护。ARY 仅接受 Organizer 同意展示的部分数据（如题目信息、排名信息等），而 Organizer 的私密数据（如测试数据集、Harness 评分标准等）留着 Organizer 自己的私有环境里，完全不流入 ARY。

（3）Organizer 的私密数据由 Organizer 提供的 runner（仅拉取/返回作用），把 Rider 的数据/对话记录拉取到 Organizer 自己的环境里进行测试，然后只将结果返回 ARY，完全不暴露私密数据。